

Trabajo Práctico Grupal

Introducción a Bases de Datos

Motivación

Este trabajo práctico tiene como objetivo integrar los distintos paradigmas y herramientas de gestión de datos abordados a lo largo de la materia. A través de un dominio de negocio real que el grupo deberá proponer, se espera que puedan evaluar qué tecnología es más adecuada para cada tipo de necesidad y justificar sus decisiones.

El entregable principal es un informe que detalle cómo se resolvió cada punto de no más de 15 páginas, incluyendo la descripción de los flujos implementados, snippets de código y los casos de uso relevados. Además del informe, se esperan los archivos de soporte: scripts SQL, notebooks de Python y cualquier configuración necesaria para reproducir el entorno.

Etapas 1: Implementación del Modelo Relacional

Esta etapa se enfoca en modelar un dominio de negocio real e implementarlo en PostgreSQL. El grupo deberá proponer un dominio que sea suficientemente rico para sustentar las etapas siguientes del trabajo. Se puede seleccionar un dominio visto en la materia o algún dominio que sea de su interés y del cual cuenten con el modelo de entidad relación correspondiente (ejemplos: taller mecánico, alquiler de embarcaciones, plataforma de streaming, sistema de turnos médicos, gestión de eventos culturales, etc.). Si bien la elección es libre, se debe informar a los docentes sobre el dominio elegido y en cualquier caso debe estar debidamente justificada.

1.1 Elección del Modelo

A partir del Diagrama Entidad-Relación (DER) del dominio elegido, justificando en cada caso:

- La elección del dominio y su relevancia.
- Las decisiones de diseño tomadas (qué se incluyó, que se agregó, qué se dejó fuera y por qué).
- Incluyan en el informe una descripción de cada constraint de negocio.

1.2 Implementación en PostgreSQL

A partir del DER, implementen el modelo en PostgreSQL. El script de creación debe incluir:

- Creación de todas las tablas con sus tipos de datos apropiados.
- Definición de claves primarias (PK) y claves foráneas (FK).

- Las constraints de negocio implementados u otras restricciones (por ejemplo: un precio no puede ser negativo, una fecha de fin no puede ser anterior a la de inicio, un stock no puede superar cierto límite, etc.).
- NO INCLUIR creación de índices en este paso

1.3 Poblado de Datos

- Generen un script de INSERT para poblar la base de datos con un volumen considerable de información que refleje escenarios realistas del dominio. El criterio mínimo es:
 - Al menos 5000 registros en las tablas principales.
 - Datos coherentes entre tablas (sin violaciones de FK, con distribuciones plausibles).
 - Variedad suficiente para que las consultas de las etapas siguientes sean significativas (evitar que todos los registros tengan los mismos valores en los campos relevantes).
- Nota: Piensen el poblado como si fuera un snapshot real del sistema en producción: distintos estados, fechas variadas, clientes con distintos comportamientos, etc.*
- Desarrollen, con SQL y usando funciones de agrupación, consultas para validar la consistencia de los datos cargados. Generar salidas del tipo de las presentadas en la práctica al calcular los estadísticos de una tabla, tanto para datos numéricos como categóricos.

Etapla 2: SQL Avanzado

En esta etapa deberán explorar funcionalidades avanzadas de SQL aplicadas sobre el modelo implementado en la Etapa 1. El objetivo es ir más allá de las consultas básicas y aprovechar capacidades analíticas del motor relacional.

2.1 Funciones de Ventana

Propongan al menos dos consultas que utilicen funciones de ventana para resolver preguntas de negocio que no sean trivialmente resolubles con GROUP BY. Ejemplos orientativos: rankear clientes por volumen de compras dentro de cada categoría, calcular relaciones entre registros sucesivos, identificar el primer o último evento por entidad.

Para cada consulta deberán:

- Describir la pregunta de negocio que responde.
- Justificar por qué una función de ventana es la herramienta adecuada.
- Mostrar y explicar brevemente el resultado.

2.2 Funciones estadísticas

Implementen tres consultas SQL para devolver los estadísticos vistos en clase para una tabla cualquiera. Deben informarse al menos los siguientes:

- C1: Para cada columna: cantidad total de filas, cantidad y porcentaje de filas con valor no nulo, cantidad de valores diferentes.
- C2: Para cada columna considerada numérica: desvío standard, mínimo, P05, primer cuartil, mediana, promedio, tercer cuartil, P95, máximo, cantidad y porcentaje de ceros, cantidad y porcentaje de valores negativos, cantidad de outliers.
- C3: Para cada columna considerada categórica: la frecuencia y el porcentaje de los hasta 10 valores más frecuentes (de mayor a menor frecuencia), y del resto.

Por otro lado, implementen al menos una consulta que utilice Common Table Expressions (WITH).

2.3 Análisis de Performance

- Seleccionen una consulta compleja de las implementadas anteriormente y analicen su plan de ejecución con EXPLAIN / EXPLAIN ANALYZE.
- Propongan una solución para mejorar el costo de ejecución, ya sea mediante creación de índices y/o implementación diferente de la consulta.
- Analicen comparativamente los costos y expliquen las diferencias de los planes.

Etapa 3: Procesamiento con Spark

En esta etapa procesarán los datos del dominio usando Apache Spark a través de PySpark. Deberán trabajar con los datos generados en la Etapa 1 (pueden exportarlos a CSV o generarlos nuevamente con Faker) y construir pipelines de procesamiento que serían impracticables o muy lentos en un motor SQL convencional.

3.1 MapReduce con Spark RDDs

Utilizando la API de RDDs de PySpark, implementen al menos tres procesamientos MapReduce sobre datos del dominio. Para cada uno deberán:

1. Describir la pregunta de negocio que se está respondiendo.
2. Explicitar la Fase de Map: qué transformación se aplica y qué estructura (clave, valor) se genera.
3. Explicitar la Fase de Reduce: qué operación de agregación se realiza y sobre qué claves.
4. Mostrar el resultado final con una impresión clara y comentada.

Orientense a consultas que sean naturalmente paralelas, por ejemplo:

- Totales o promedios agrupados por alguna dimensión del dominio.
- Conteo de ocurrencias por categoría o estado.
- Identificación de los N elementos más frecuentes o de mayor valor.

***Nota:** Presten atención a la diferencia entre transformaciones (lazy) y acciones. En el informe comenten cuándo Spark ejecuta realmente el cómputo y por qué.*

Etapa 4: Persistencia Políglota — NoSQL

Los nuevos requerimientos del negocio exceden las prestaciones del modelo relacional. En esta etapa deberán pensar el sistema como una arquitectura de persistencia políglota: diferentes tecnologías de almacenamiento para diferentes necesidades.

A partir del dominio ya modelado, deberán identificar qué datos o patrones de acceso se beneficiarían de cada motor NoSQL y justificar sus decisiones en el informe.

4.1 Redis

Redis es un almacén de estructuras de datos en memoria, ideal para caché, contadores, colas de trabajo y datos con tiempo de vida acotado. Implementen los siguientes puntos en una notebook de Python.

4.1.1 Modelo Clave-Valor y Hashes

Identifiquen al menos tres tipos de datos del dominio que sea natural almacenar como pares clave-valor simples o como hashes. Piensen en datos que se consultan frecuentemente de forma individual (por ID), como la información de perfil de un usuario, la configuración de un servicio, el estado actual de un proceso, etc.

Implementen en código:

- La carga de estos datos en Redis (set / hset con estructuras apropiadas).
- Consultas que recuperen valores específicos (get / hget / hgetall).
- Al menos una actualización de un campo y la verificación del cambio.

En el informe expliquen por qué eligieron Redis sobre PostgreSQL para este caso y qué ventajas concretas aporta en el contexto del dominio.

4.1.2 Listas como Colas o Historial

Propongan un escenario del dominio donde una lista de Redis sea la estructura adecuada. Puede ser una cola de tareas pendientes, un historial de eventos recientes, una lista de notificaciones, un log de acciones, etc.

Implementen:

- La creación y carga de la lista con datos del dominio (rpush / lpush).
- Operaciones de consulta: lrange, llen.
- Operaciones de gestión: agregar nuevos elementos, recuperar y eliminar desde un extremo (lpop / rpop).
- Una simulación del flujo completo: cómo entran y salen elementos de la lista en el contexto del negocio.

4.1.3 TTL: Datos con Tiempo de Vida

Identifiquen datos del dominio para los cuales tenga sentido que expiren automáticamente. Piensen en tokens de sesión, reservas temporales, descuentos por tiempo limitado, alertas de stock, cachés de resultados de consultas costosas, etc.

Implementen:

- Al menos tres claves con TTL distinto, justificando el tiempo elegido para cada una.
- Un mecanismo que verifique el tiempo restante de una clave (ttl / pttl).
- La impresión de mensajes representativos que simulen el comportamiento del sistema ("La reserva X expiró", "El token del usuario Y sigue vigente por N segundos", etc.).

***Nota:** El TTL no es solo una feature técnica: elegir bien cuándo los datos deben expirar es una decisión de diseño que impacta en la consistencia y el rendimiento del sistema.*

4.2 MongoDB

MongoDB es una base de datos documental orientada a almacenar datos semi-estructurados o con esquema variable. Es especialmente útil cuando los objetos del dominio tienen atributos heterogéneos, cuando se quiere evitar joins costosos embebiendo datos relacionados, o cuando la estructura de los datos evoluciona frecuentemente.

Configuración del entorno

Deben elegir una de las siguientes opciones para correr MongoDB:

- Crear una instancia gratuita en MongoDB Atlas (nube) y conectarse desde la notebook con PyMongo.
- Levantar MongoDB con Docker en sus máquinas locales usando la imagen oficial.

Documenten en el informe los pasos seguidos para la configuración del entorno elegido, incluyendo el string de conexión utilizado (con credenciales anonimizadas) y cualquier problema encontrado.

4.2.1 Diseño de Colecciones

Propongan al menos dos colecciones de documentos que representen entidades del dominio. Discutan las siguientes decisiones de diseño:

- ¿Qué datos conviene embeber dentro del documento y cuáles referenciar por ID?
- ¿Qué campos son opcionales o pueden variar entre documentos?
- ¿Cómo difiere este diseño del modelo relacional de la Etapa 1 y por qué?

Generen al menos 100 documentos por colección, usando Faker o datos del dominio ya generados. Incluyan variedad en los documentos: distintos campos opcionales presentes, distintos valores, al menos un campo de tipo array o documento embebido.

4.2.2 Operaciones CRUD

Implementen y documenten las siguientes operaciones sobre las colecciones:

- Inserción de documentos individuales y en lote (`insert_one`, `insert_many`).
- Consultas con filtros sobre distintos campos, incluyendo operadores de comparación (`$gt`, `$lt`, `$gte`, `$lte`) y lógicos (`$and`, `$or`).
- Proyecciones para devolver solo los campos relevantes.
- Actualización de documentos (`update_one`, `update_many` con `$set`, `$push`, `$inc`).
- Eliminación condicional (`delete_one`, `delete_many`).

4.2.3 Aggregation Pipelines

Implementen al menos dos pipelines de agregación sobre los datos del dominio. Cada pipeline debe usar al menos tres etapas distintas. Oriéntense a responder preguntas de negocio relevantes, por ejemplo:

- Ranking de entidades por alguna métrica calculada.
- Agrupamiento por período de tiempo (mes, trimestre) con cálculo de totales o promedios.
- Filtrado seguido de proyección y ordenamiento.
- Uso de \$lookup para combinar datos de dos colecciones (equivalente a un JOIN).

Para cada pipeline, expliquen en el informe cada etapa y qué transformación realiza sobre los documentos.

***Nota:** Reflexionen sobre las diferencias entre resolver estas consultas en PostgreSQL, SparkSQL y MongoDB. ¿En qué casos MongoDB resulta más natural o más eficiente? ¿Cuándo no lo sería?*

Criterios de entrega

El grupo deberá entregar un único repositorio o carpeta comprimida que contenga:

- Informe en PDF con la resolución de todos los puntos, justificaciones y capturas de resultados.
- Script SQL de creación de tablas y constraints (Etapa 1).
- Script SQL poblado de datos (Etapa 1).
- Scripts SQL de las consultas avanzadas (Etapa 2).
- Notebook para MapReduce con Spark (Etapa 3.1).
- Notebook para Redis (Etapa 4.1).
- Notebook MongoDB (Etapa 4.2).
- Archivo README con instrucciones para reproducir el entorno (versiones, dependencias, cómo levantar los servicios).

Todos los archivos deben ser ejecutables de punta a punta sin modificaciones. El código debe estar comentado y el informe debe ser autocontenido: alguien que no haya participado del desarrollo debe poder entender qué se hizo y por qué.